

IS 365 - Practical Assignment

Full-Stack Pipeline for Deploying a Self-Hosted LLM Application

Instructions:

Form a group of 5–7 students, then attempt the following assignment.

1. Assignment

Building and Deploying a Self-Hosted LLM Application

2. Background

Modern AI applications are not built by simply calling a model. In real-world industry environments, an AI system normally has several connected components, including:

- a server or development environment;
- a Python virtual environment;
- a locally hosted Large Language Model or Small Language Model;
- an API backend that receives requests from users;
- a frontend interface for interaction;
- logging, testing, and error handling;
- deployment preparation and documentation.

Focus: to build a complete AI application pipeline using a locally hosted LLM.

3. Use Case

You are required to build a simple **University Student Support Assistant**.

The assistant should help students ask questions about university services such as:

- course registration;
- examination rules;
- library services;
- ICT support;
- hostel application;
- fee payment;
- academic calendar;
- student conduct.

The system does not need to be perfect. The main focus is to demonstrate understanding of the full deployment pipeline.

4. Main Objective

The main objective is to design, implement, test, and document a complete self-hosted LLM application pipeline similar to what may be required in an industry production environment.

5. Specific Learning Outcomes

By completing this assignment, you should be able to:

1. Prepare a local development environment for an AI application.
2. Install and run a local LLM
3. Create a FastAPI backend that communicates with the local LLM.
4. Build a simple frontend interface.
5. Connect the frontend, backend, and LLM into one working pipeline.
6. Test API endpoints using a browser, Swagger UI, or Python scripts.
7. Implement basic logging and error handling.
8. Explain the difference between prototype deployment and production deployment.
9. Prepare technical documentation and evidence of implementation.

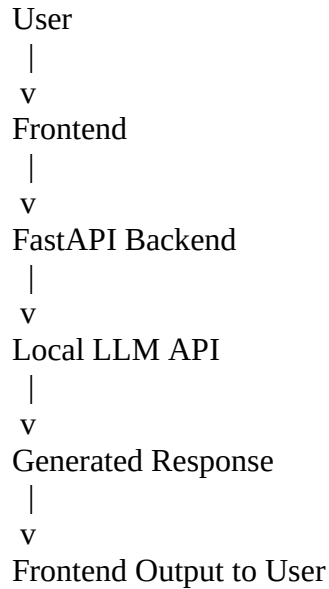
6. Required Tools

Each student or group should use the following tools:

Component	Recommended Tool
Operating system	Windows, Linux, or macOS
Code editor	Visual Studio Code
Programming language	Python 3.10 or above
Virtual environment	Python venv
Backend API	FastAPI, Flask, Django
Web server	Uvicorn, Gunicorn, Granian, Hypercom, Daphne
Local LLM serving	Ollama or any
Model	llama3.2:1b, phi3, or any lightweight local model
Frontend	React, Vue, Angular, Streamlit, Gradio
API testing	Browser, Swagger UI, curl, Postman, or Python requests
Version control	Git/GitHub, optional but recommended

7. Expected System Architecture

Your final system should follow this logical flow:

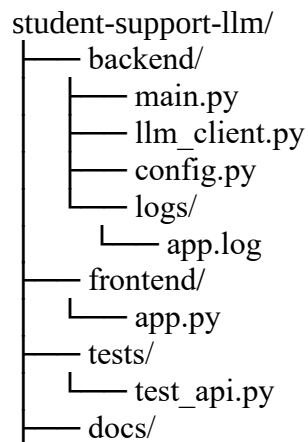


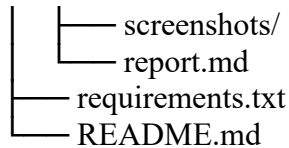
The system should also include:

- Configuration File
- Logging
- Error Handling
- Testing Script
- README Documentation

8. Recommended Project Structure

Each group should organise the project as follows:





9. Submission

Provide evidence for the following tasks:

Task 1: Environment Setup

- Screenshot showing the activated virtual environment.
- Screenshot showing successful installation of libraries.

Task 2: Installation and Running of a Local LLM

- Screenshot showing the model being pulled.
- Screenshot showing the model running.
- Screenshot showing a successful API response.

Task 3: The Developed FastAPI Backend

- Screenshot showing FastAPI running.
- Screenshot of /docs.
- Screenshot of a successful /health response.
- Screenshot of a successful /ask response.

Task 4: The Developed Frontend

- Screenshot showing the frontend.
- Screenshot showing a question-and-answer interaction.

Task 5: The Created API Test Script

- Screenshot showing successful test output.

Task 6: The Improved Prompt

- Show the original prompt.
- Show the improved prompt.
- Compare one response before and after prompt improvement.

Task 7: Basic Error Handling

Your system should handle the following situations:

Situation	Expected Behaviour
Backend is not running	Frontend should show connection error
model is not running	Backend should return clear error
Empty question	Frontend should ask user to enter a question
Slow response	Frontend should show loading/spinner message

Evidence required:

- Screenshot or explanation showing how each error is handled.

Task 8: Logging

The backend should record:

- received questions;
- generated answers;
- errors;
- timestamp of each interaction.

Evidence required:

- Screenshot or copied extract of the log file.

Task 9: Industry Production Reflection

Write a short reflection by answering the following questions:

1. What are the main components of your deployed LLM system?
2. Why is FastAPI useful in this pipeline?
3. What role does your chosen LLM model play?
4. What role does the frontend play?
5. What is the difference between running the model locally and using an external API?
6. What security risks may exist if this system is deployed in an organisation?
7. What improvements would be needed before deploying this system in production?
8. How would you monitor the system in real-world use?
9. How would you protect sensitive student information?
10. What challenges did you face during implementation?

10. Bonus marks for Optional Extension

Add one or more of the following features:

Option A: Add Document-Based Question Answering

Allow the user to upload a .txt or .md file and ask questions from it.

Option B: Add Simple RAG

Store a small university FAQ document and retrieve relevant sections before sending the prompt to the LLM.

Option C: Add Docker

Containerize the backend using a Dockerfile.

Option D: Add Authentication

Add a simple login page or API key check.

Option E: Add Response Evaluation

Ask users to rate the answer as:

Good / Average / Poor

Save the feedback in a file.

11. Final Deliverables

Each group must submit:

1. Source code folder or GitHub repository link.
2. README.md with setup and running instructions.
3. Screenshots showing:
 - virtual environment setup;
 - LLM model running;
 - FastAPI backend running;
 - Swagger /docs;
 - frontend;
 - sample question and answer;
 - test script output;
 - log file.
4. Short technical report, 4–6 pages.
5. Reflection answers.

12. Suggested Report Structure

The report should follow this structure:

1. Introduction
2. System Use Case
3. Tools and Technologies Used
4. System Architecture
5. Implementation Steps
6. Testing and Results
7. Challenges Encountered
8. Production Readiness Discussion
9. Conclusion
10. Appendix: Screenshots and Code Snippets

15. Submission Instructions

Submit the assignment as a compressed folder or GitHub repository containing:

```
student-support-llm/  
├── backend/  
├── frontend/  
├── tests/  
├── docs/  
├── requirements.txt  
└── README.md
```

The report should be submitted as a PDF file.

16. Note

You are reminded that the purpose of this assignment is not to build the most intelligent chatbot. The purpose is to understand the complete pipeline for deploying an LLM-based system.

Each group should therefore be able to explain:

- where the model is running;
- how the frontend communicates with the backend;
- how the backend communicates with the model;
- how requests and responses move through the system;
- how the system can be tested;
- what should be improved before real production deployment.